Importations

In [10]:
```
# package importation
    # data
import pandas as pd
import numpy as np

    # plot
import matplotlib.pyplot as plt
from matplotlib import cm                       #Colormap
import matplotlib.patches as mpatches
import matplotlib.lines as mlines
import matplotlib.transforms as mtransforms
import matplotlib.gridspec as gridspec          #Subplot in subplot
from matplotlib.ticker import MaxNLocator

import seaborn as sns

import scipy
from scipy.stats import ttest_ind as ttest


    # data treatment
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from collections import Counter
```

In [25]:
```
sns.set_style("darkgrid")

DATA_PATH = "../data/"
OUTPUT_PATH = './output/'
```

In [11]:
```
# data import
df_syn = pd.read_csv(DATA_PATH + "synthetic_data.txt", sep=";")
df_syn.head()
```

Out[11]:

	ID	ID per year	Year	Dum_Default	Total equity total assets	Total reg cap ratio	Liquid assets total assets	Deposits mm funding growth	Expenses rev	Net int margin
0	13215466	132154662014	2014	0	14.910683	18.282517	3.077498	1616.302825	56.023078	3.710629
1	21776652	217766522002	2002	0	13.900000	19.100000	5.600000	60.000000	67.500000	6.200000
2	24093640	240936402012	2012	0	9.279741	14.820007	6.387157	-50.020364	53.888990	4.249898
3	18010425	180104252005	2005	0	10.020848	12.377526	4.335830	-37.427293	70.134845	3.835830
4	3606768	36067682009	2009	0	9.466163	21.222226	5.923254	146.421381	98.272408	3.207873

5 rows × 24 columns

In [44]:
```
l_id_var = list(df)[3:]
l_exo_var = list(df)[4:]
```

Basic descriptive statistics: about ones

In [15]:
```
df.describe()
```

Out[15]:

	ID	ID per year	Year	Dum_Default	Total equity total assets	Total reg cap ratio	Liquid assets total assets	Deposi fu g
count	5.971000e+04	5.971000e+04	59710.000000	59710.000000	59710.000000	59710.000000	59710.000000	5.9710(
mean	1.916299e+07	1.916299e+11	2008.699833	0.008374	10.879297	17.468977	8.345919	7.23855
std	1.041983e+07	1.041983e+11	4.434202	0.091125	4.113871	13.117105	6.641299	7.28944
min	1.711000e+03	1.711201e+07	2000.000000	0.000000	-4.831861	-7.991473	0.009520	-9.91477
25%	1.088812e+07	1.088812e+11	2005.000000	0.000000	8.873855	12.876381	4.073877	1.41933
50%	1.935507e+07	1.935507e+11	2009.000000	0.000000	10.144325	15.082529	6.616483	5.49993
75%	2.651627e+07	2.651627e+11	2012.000000	0.000000	11.872764	18.742663	10.511903	1.11656
max	5.791253e+07	5.791253e+11	2018.000000	1.000000	89.593746	873.513220	97.000000	1.18067

8 rows × 24 columns

In [16]:
```
df["total_assets"] = np.exp(df["log_Total assets"])
```

In [17]:
```
df.describe()
```

Out[17]:

	ID	ID per year	Year	Dum_Default	Total equity total assets	Total reg cap ratio	Liquid assets total assets	Deposi fu g
count	5.971000e+04	5.971000e+04	59710.000000	59710.000000	59710.000000	59710.000000	59710.000000	5.9710(
mean	1.916299e+07	1.916299e+11	2008.699833	0.008374	10.879297	17.468977	8.345919	7.23855
std	1.041983e+07	1.041983e+11	4.434202	0.091125	4.113871	13.117105	6.641299	7.28944
min	1.711000e+03	1.711201e+07	2000.000000	0.000000	-4.831861	-7.991473	0.009520	-9.91477
25%	1.088812e+07	1.088812e+11	2005.000000	0.000000	8.873855	12.876381	4.073877	1.41933
50%	1.935507e+07	1.935507e+11	2009.000000	0.000000	10.144325	15.082529	6.616483	5.49993
75%	2.651627e+07	2.651627e+11	2012.000000	0.000000	11.872764	18.742663	10.511903	1.11656
max	5.791253e+07	5.791253e+11	2018.000000	1.000000	89.593746	873.513220	97.000000	1.18067

8 rows × 25 columns

In [18]:
```
df[df["Dum_Default"]==1].describe()
```

Out[18]:

	ID	ID per year	Year	Dum_Default	Total equity total assets	Total reg cap ratio	Liquid assets total assets	Deposits mm funding growth	
count	5.000000e+02	5.000000e+02	500.000000	500.0	500.000000	500.000000	500.000000	500.000000	
mean	7.155121e+06	7.155121e+10	2009.044000	1.0	4.815514	7.334780	12.075744	23.849180	
std	8.556151e+06	8.556151e+10	1.687531	0.0	2.976829	4.316640	6.650574	1873.442982	
min	4.092400e+05	4.092402e+09	2003.000000	1.0	-4.831861	-7.991473	0.470262	-3951.011937	-
25%	1.206223e+06	1.206223e+10	2008.000000	1.0	2.688418	4.627381	7.747546	-1006.538738	
50%	2.415076e+06	2.415076e+10	2009.000000	1.0	4.678487	7.181885	10.932494	-242.582365	
75%	9.630228e+06	9.630228e+10	2010.000000	1.0	6.877658	9.983198	15.365209	662.651574	
max	3.845315e+07	3.845315e+11	2017.000000	1.0	14.708994	27.320329	47.948359	13297.392926	5

8 rows × 25 columns

In [19]:
```
df[df["Dum_Default"]==1][["total_assets"]].describe()
```

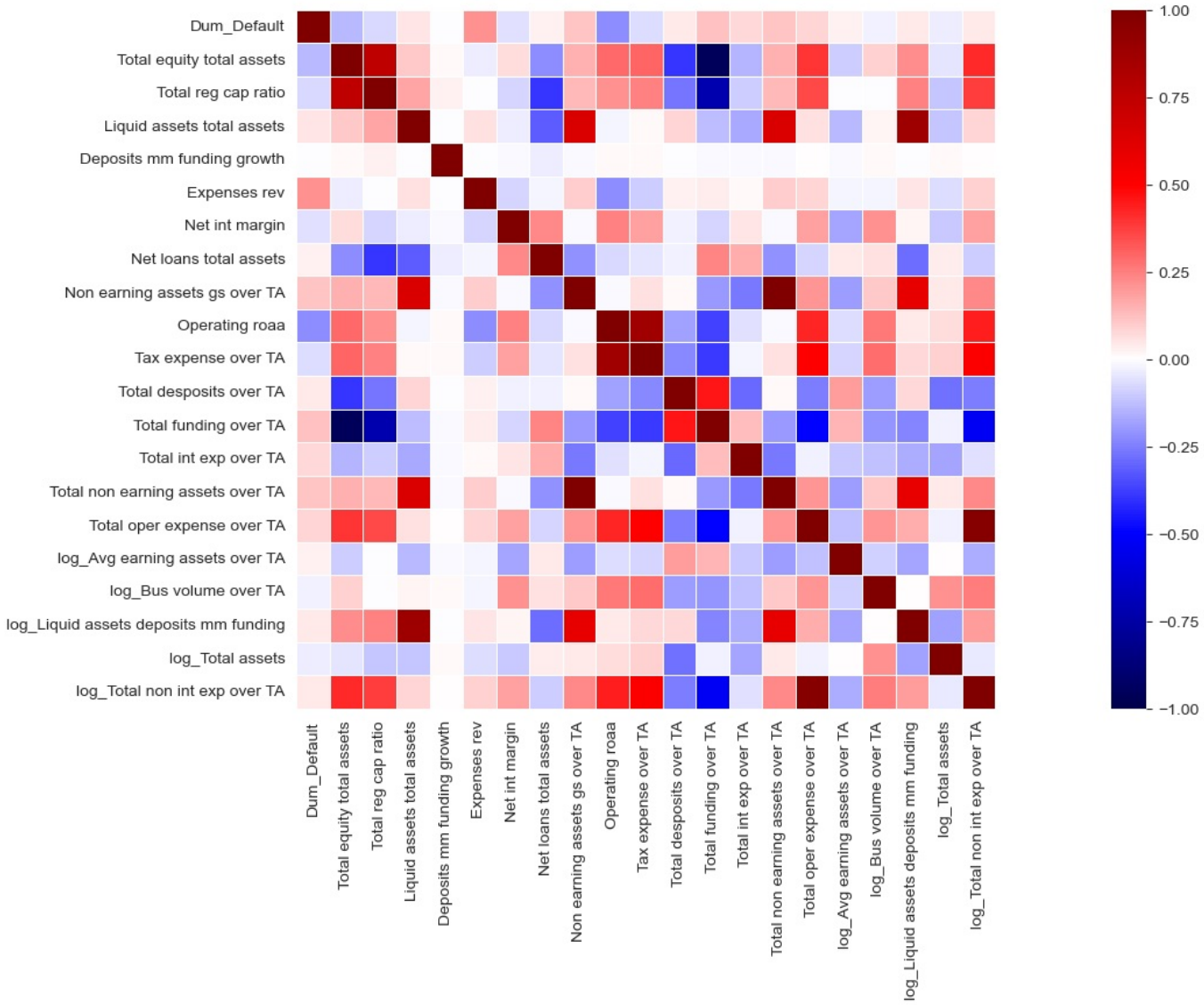
Out[19]:

	total_assets
count	5.000000e+02
mean	2.930269e+08
std	4.209852e+08
min	1.499964e+07
25%	1.093939e+08
50%	1.812926e+08
75%	3.071060e+08
max	5.131917e+09

Correlation analysis

In [21]:

```
"""
CORRELATION ANALYSIS
- heatmap
"""

#Set fig size
plt.rcParams['figure.figsize'] = (20,10)

#Data to plot
df_plot = df[l_exo_var]
corr = df_plot.corr()

#Set axes
f,ax = plt.subplots(figsize=(20, 10))

f.subplots_adjust(top = 0.95, bottom = 0.35,hspace=0.4, right=1, left=0)

#Range of color for the hitmap
cmap = cm.seismic

#Plotting cmd
sns.heatmap(corr, cmap=cmap, center=0,
            square=True, vmin=-1, linewidths=.5)

# plt.savefig(OUTPUT_PATH + 'corr_var.png')

plt.show()
```



In [45]:
```
df_plot = df[["Dum_Default"]+l_exo_var]
corr = df_plot.corr()

corr.iloc[0].sort_values(ascending=False)
```

Out[45]:
```
Dum_Default                          1.000000
Expenses rev                         0.217057
Total funding over TA                0.119707
Total non earning assets over TA     0.111010
Non earning assets gs over TA        0.111010
Total oper expense over TA           0.084350
Total int exp over TA                0.077174
Liquid assets total assets           0.051609
log_Liquid assets deposits mm funding    0.044359
log_Total non int exp over TA        0.043796
Total desposits over TA              0.042192
log_Avg earning assets over TA       0.028856
Net loans total assets               0.023677
Deposits mm funding growth          -0.000910
total_assets                        -0.006142
log_Bus volume over TA              -0.023851
log_Total assets                    -0.031475
Net int margin                      -0.062331
Tax expense over TA                 -0.068564
Total reg cap ratio                 -0.070997
Total equity total assets           -0.135452
Operating roaa                      -0.223604
Name: Dum_Default, dtype: float64
```

SMOTE: before and after

We look into the effect of the smote procedure on data distribution. To do so we will compare mathematical expectation and variance of the train data set before and after the SMOTE procedure.

Precisely, we will will compare those quantities for the full train dataset and the dataset restrained to default banks (those for which new points are created).

We will lead this comparison using statistical inference tests.

describe the test

for the full dataset

In [46]:
```
# Defining features and labels matrices
X = df[list(df)[4:]]
y = df[list(df)[3]]

# Splitting into trainnig and testing samples
X_train, X_test, y_train, y_test = \
train_test_split(X,y,test_size=0.3,random_state=42)

print('We have ',sum(y_test),' default in the test sample.')

# Applying SMOTE procedure on training data
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print('Before smote: ', Counter(y_train))
print('After smote: ', Counter(y_train_smote))
```

```
We have  151  default in the test sample.
Before smote:  Counter({0: 41448, 1: 349})
After smote:  Counter({0: 41448, 1: 41448})
```

In [47]:
```
X_train.shape, X_train_smote.shape
```

Out[47]:
```
((41797, 21), (82896, 21))
```

In [48]:
```
# test with the first column

n_b = X_train.shape[0]
n_a = X_train_smote.shape[0]


before = X_train.iloc[:,0]
after = X_train_smote.iloc[:,0]

m_b = np.mean(before)
m_a = np.mean(after)

v_b = np.var(before)
v_a = np.var(after)

S = (n_b * m_b + n_a * m_a)/(n_a+n_b-2)


T = (m_b - m_a)/(S*(1/n_b+1/m_a))

deg_f = n_a+n_b-2
print(deg_f)
```

```
124691
```

In [49]:
```
# test with the first column

n_b = X_train.shape[0]
n_a = X_train_smote.shape[0]
t_stat = 3.291 # confidence 0.9995 and degree of freedom inf
               # total confidence: 1 per thousand in a bilateral test
# t_stat = 2.576  # confidence 0.99

f_stat = 1 # 1percent risk in a unilateral test


n_var = X_train.shape[1]

for i in range(n_var):
    # isolating the variable
    before = X_train.iloc[:,i]
    after = X_train_smote.iloc[:,i]

    # means
    m_b = np.mean(before)
    m_a = np.mean(after)

    # variances
    v_b = np.var(before)
    v_a = np.var(after)
```

```
    # "joint variance"
    S = (n_b * m_b + n_a * m_a)/(n_a+n_b-2)

    # T stat for means comparison
    T = (m_b - m_a)/(S*(1/n_b+1/m_a))

    # F stat for variance comparison
    F = v_b/v_a


    if T>t_stat:
        print("diff in means: ", list(X_train)[i])
        print(T)

    if F>f_stat:
        print("diff in variance: ",list(X_train)[i])
        print(F)
```

```
diff in means:  Total reg cap ratio
4.48912345426285
diff in variance:  Total reg cap ratio
1.437152005572411
diff in variance:  Liquid assets total assets
1.0706968337855585
diff in means:  Deposits mm funding growth
2659.8765556450344
diff in variance:  Deposits mm funding growth
1.9831901260281155
diff in variance:  Net int margin
1.2103476103577602
diff in variance:  Net loans total assets
1.5363152901593142
diff in means:  Operating roaa
6.602552136854072
diff in variance:  Total desposits over TA
1.389502087608516
diff in variance:  Total int exp over TA
1.0672005135480644
diff in variance:  Total oper expense over TA
1.0004990402721479
diff in variance:  log_Avg earning assets over TA
1.025388331940239
diff in variance:  log_Bus volume over TA
1.8389247526683716
diff in variance:  log_Liquid assets deposits mm funding
1.225831016795654
diff in variance:  log_Total assets
1.363490920483451
diff in variance:  log_Total non int exp over TA
1.1100183794250427
diff in means:  total_assets
28302.121898207468
diff in variance:  total_assets
1.9787369461312474
```

for the class 1

In [50]:
```
X_train["default"] = y_train
X_train_smote["default"] = y_train_smote

df_one = X_train[X_train["default"]==1]
df_one_smote = X_train_smote[X_train_smote["default"]==1]
```

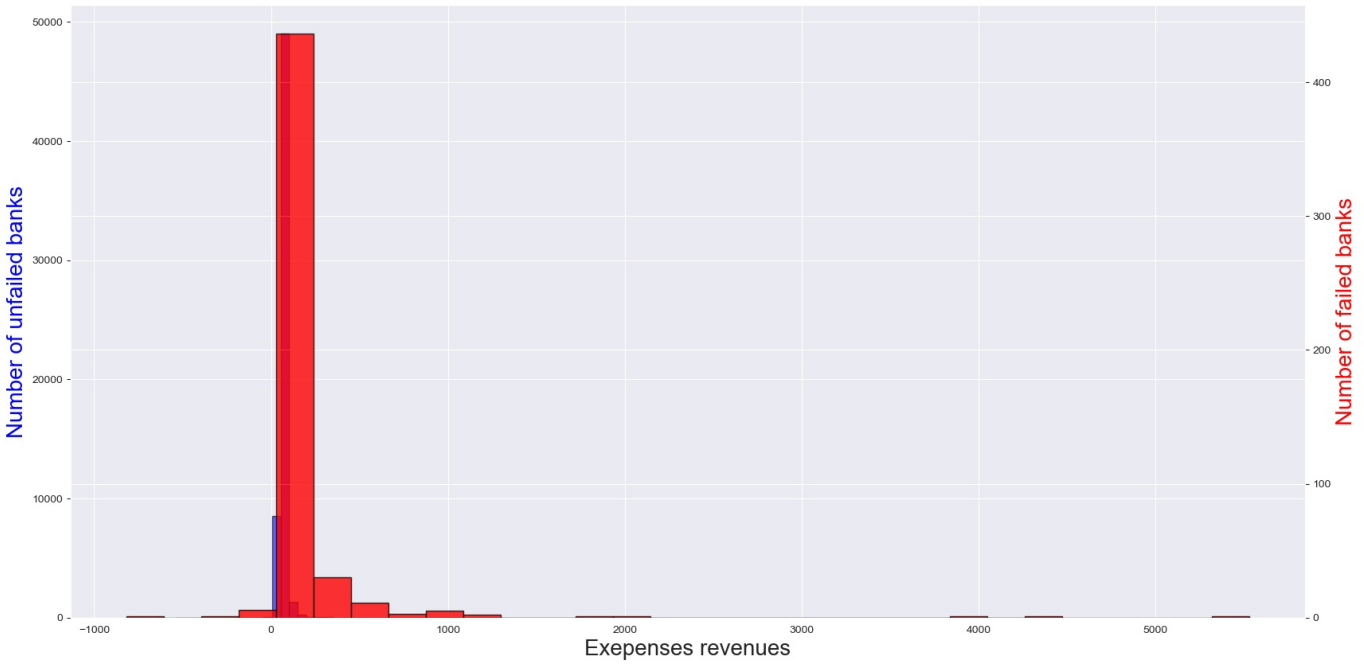
In [51]:
```
# test with the first column

n_b = df_one.shape[0]
n_a = df_one_smote.shape[0]
t_stat = 3.291 # confidence 0.9995 and degree of freedom inf
                # total confidence: 1 per thousand in a bilateral test

f_stat = 1 # 1percent risk in a unilateral test


n_var = df_one.shape[1] - 1

for i in range(n_var):
    # isolating the variable
    before = df_one.iloc[1:,i]
    after = df_one_smote.iloc[1:,i]

    # means
    m_b = np.mean(before)
```

```
    m_a = np.mean(after)

    # variances
    v_b = np.var(before)
    v_a = np.var(after)

    # "joint variance"
    S = (n_b * m_b + n_a * m_a)/(n_a+n_b-2)

    # T stat for means comparison
    T = (m_b - m_a)/(S*(1/n_b+1/m_a))

    # F stat for variance comparison
    F = v_a/v_b


    if T>t_stat:
        print("diff in means: ", list(df_one)[i+1])
        print(T, m_a, m_b)

    if F>f_stat:
        print("diff in variance: ",list(df_one)[i+1])
        print(F)
```

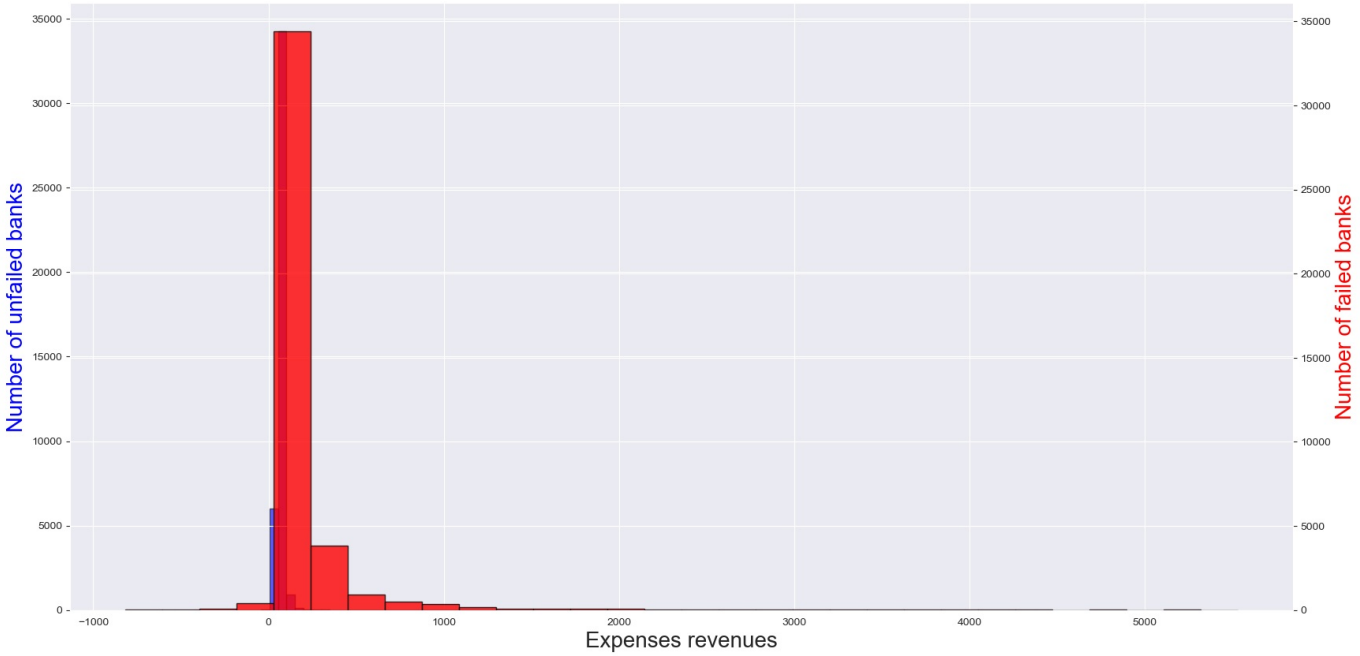
```
diff in means:  Expenses rev
23.889176491829996 -6.73883171259851 16.02922927328211
diff in means:  default
6.537841623165751 265147375.33285373 270115421.17987263
```

In [52]:

```
# BEFORE SMOTE

sns.set_style("darkgrid")

fig, ax1 = plt.subplots()


ax1.set_xlabel('Exepenses revenues',fontsize=20)
ax1.hist(df[(df['Dum_Default']==0)]['Expenses rev']
         , color='blue', edgecolor='black', bins=int(100), alpha = 0.6)
ax1.set_ylabel('Number of unfailed banks',fontsize=20, color='b')


ax2 = ax1.twinx()  # instantiate a second axes that shares the same x-axis

ax2.set_ylabel('Number of failed banks',fontsize=20, color='r')  # we already handled the x-label with ax1
ax2.hist(df[(df['Dum_Default']==1)]['Expenses rev']
         , color='r', edgecolor='black', bins=int(30), alpha = 0.8)


#plt.title('TE/TA - Distribution, all banks',fontsize = 22)


#plt.savefig(OUPUT_PATH + 'distribution_TE_over_TA_us.png')


plt.show()
```

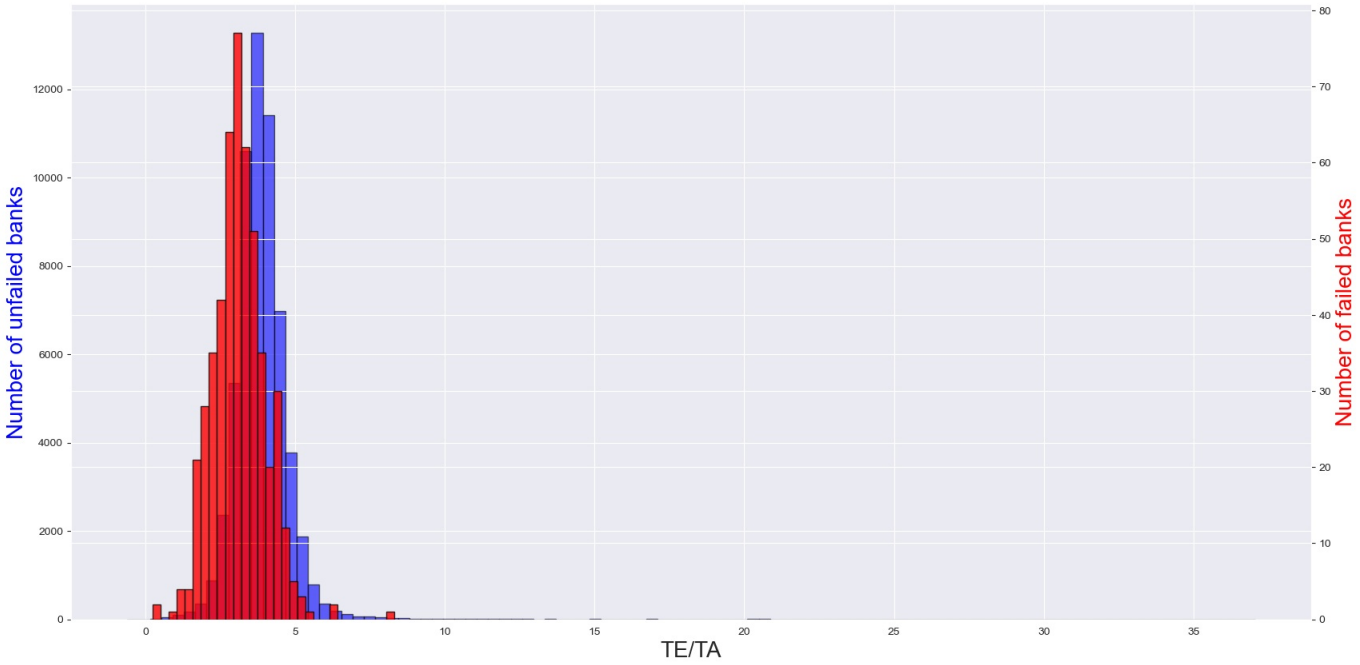


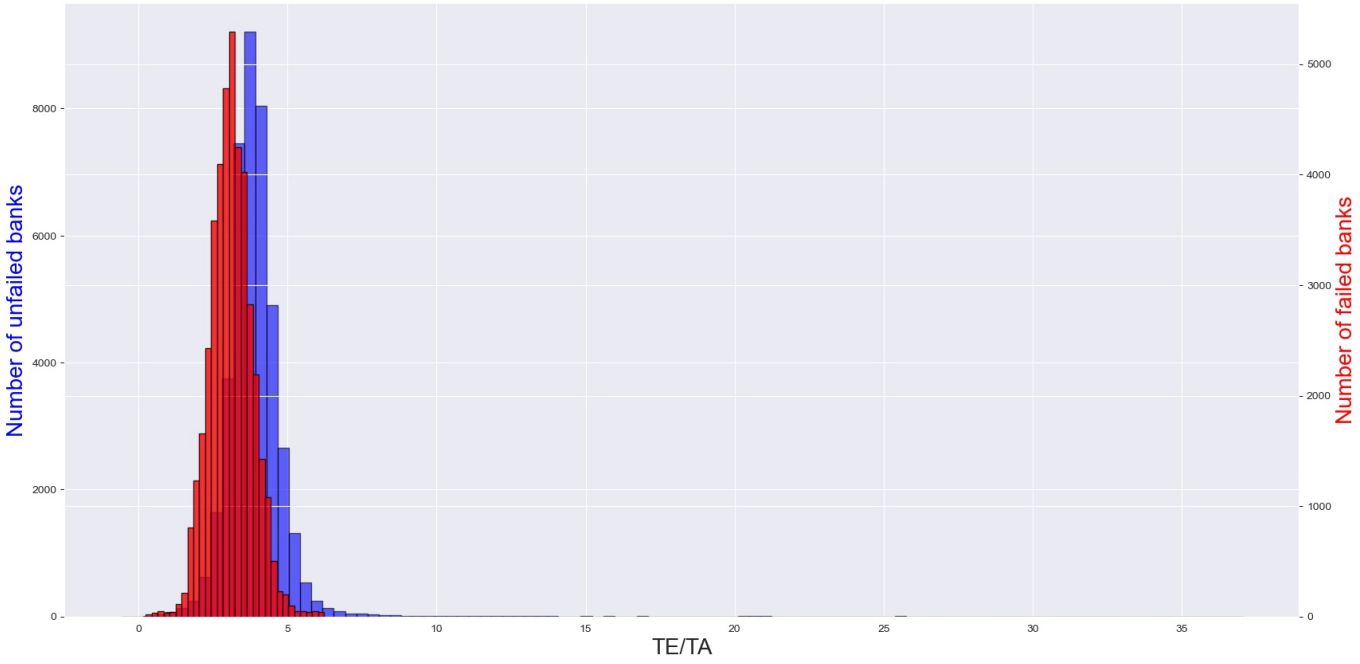
In [53]:
```
# AFTER SMOTE

sns.set_style("darkgrid")

fig, ax1 = plt.subplots()


ax1.set_xlabel('Expenses revenues',fontsize=20)
ax1.hist(X_train_smote[(X_train_smote['default']==0)]['Expenses rev']
         , color='blue', edgecolor='black', bins=int(100), alpha = 0.6)
ax1.set_ylabel('Number of unfailed banks',fontsize=20, color='b')


ax2 = ax1.twinx()  # instantiate a second axes that shares the same x-axis

ax2.set_ylabel('Number of failed banks',fontsize=20, color='r')  # we already handled the x-label with ax1
ax2.hist(X_train_smote[(X_train_smote['default']==1)]['Expenses rev']
         , color='r', edgecolor='black', bins=int(30), alpha = 0.8)


#plt.title('TE/TA - Distribution, all banks',fontsize = 22)


#plt.savefig(OUPUTPATH + 'distribution_TE_over_TA_us.png')


plt.show()
```



In [54]:
```
# Distribution of the main variable: TE/TA

sns.set_style("darkgrid")

fig, ax1 = plt.subplots()


ax1.set_xlabel('TE/TA',fontsize=20)
ax1.hist(df[(df['Dum_Default']==0)]['Net int margin']
         , color='blue', edgecolor='black', bins=int(100), alpha = 0.6)
ax1.set_ylabel('Number of unfailed banks',fontsize=20, color='b')


ax2 = ax1.twinx()  # instantiate a second axes that shares the same x-axis

ax2.set_ylabel('Number of failed banks',fontsize=20, color='r')  # we already handled the x-label with ax1
ax2.hist(df[(df['Dum_Default']==1)]['Net int margin']
         , color='r', edgecolor='black', bins=int(30), alpha = 0.8)


#plt.title('TE/TA - Distribution, all banks',fontsize = 22)


#plt.savefig(path_output + 'distribution_TE_over_TA_us.png')


plt.show()
```

```
# Distribution of the main variable: TE/TA

sns.set_style("darkgrid")

fig, ax1 = plt.subplots()


ax1.set_xlabel('TE/TA',fontsize=20)
ax1.hist(X_train_smote[(X_train_smote['default']==0)]['Net int margin']
         , color='blue', edgecolor='black', bins=int(100), alpha = 0.6)
ax1.set_ylabel('Number of unfailed banks',fontsize=20, color='b')


ax2 = ax1.twinx()  # instantiate a second axes that shares the same x-axis

ax2.set_ylabel('Number of failed banks',fontsize=20, color='r')  # we already handled the x-label with ax1
ax2.hist(X_train_smote[(X_train_smote['default']==1)]['Net int margin']
         , color='r', edgecolor='black', bins=int(30), alpha = 0.8)


#plt.title('TE/TA - Distribution, all banks',fontsize = 22)


#plt.savefig(path_output + 'distribution_TE_over_TA_us.png')


plt.show()
```





Significant difference in mean and variance: self-impact

- we test the significancy of the difference in mean and variance for each variable:
  - Student test for means
  - Fisher test for variances
- In each case: we look into the means and variances of a variable for the sample in which Dum_Default scores 1 and 0

$\Rightarrow$ Those tests tell us about the relevance of each variable ability to split the sample when looking into the default variable.

Difference in mean

In [55]:
```
for var in l_exo_var:
    a = df[df["Dum_Default"]==0][var]
    b = df[df["Dum_Default"]==1][var]
    iz = ttest(a,b)
    print(var," : ",iz[0], " et pvalue : ", np.round(iz[1],2))
```

```
Total equity total assets  :  33.40576113636851  et pvalue :  0.0
Total reg cap ratio  :  17.392260504517406  et pvalue :  0.0
Liquid assets total assets  :  -12.627632689504921  et pvalue :  0.0
Deposits mm funding growth  :  0.2222448123765372  et pvalue :  0.82
Expenses rev  :  -54.33383768193925  et pvalue :  0.0
Net int margin  :  15.260416787322576  et pvalue :  0.0
Net loans total assets  :  -5.787088327308402  et pvalue :  0.0
Non earning assets gs over TA  :  -27.29425721983769  et pvalue :  0.0
Operating roaa  :  56.05757059890101  et pvalue :  0.0
Tax expense over TA  :  16.793197632220096  et pvalue :  0.0
Total desposits over TA  :  -10.318888040170284  et pvalue :  0.0
Total funding over TA  :  -29.462627254481667  et pvalue :  0.0
Total int exp over TA  :  -18.91409821040647  et pvalue :  0.0
Total non earning assets over TA  :  -27.294257232924714  et pvalue :  0.0
Total oper expense over TA  :  -20.684873924955763  et pvalue :  0.0
log_Avg earning assets over TA  :  -7.0540032395129835  et pvalue :  0.0
log_Bus volume over TA  :  5.829649435084043  et pvalue :  0.0
log_Liquid assets deposits mm funding  :  -10.849966119714532  et pvalue :  0.0
log_Total assets  :  7.694909839973813  et pvalue :  0.0
log_Total non int exp over TA  :  -10.711950450516152  et pvalue :  0.0
total_assets  :  1.5007570869532147  et pvalue :  0.13
```

Difference in variance

In [56]:
```
#define F-test function
def f_test(x, y):
    x = np.array(x)
    y = np.array(y)
    f = np.var(x, ddof=1)/np.var(y, ddof=1) #calculate F test statistic
    dfn = x.size-1 #define degrees of freedom numerator
    dfd = y.size-1 #define degrees of freedom denominator
    p = 1-scipy.stats.f.cdf(f, dfn, dfd) #find p-value of F test statistic
    return f, p
```

In [57]:
```
for var in l_exo_var:
    a = df[df["Dum_Default"]==0][var]
    b = df[df["Dum_Default"]==1][var]
    iz = f_test(a,b)
    print(var," : ",iz[0], " et pvalue : ", np.round(iz[1],2))
```

```
Total equity total assets  :  1.8821891645425162  et pvalue :  0.0
Total reg cap ratio  :  9.256494739971826  et pvalue :  0.0
Liquid assets total assets  :  0.9945276570640089  et pvalue :  0.54
Deposits mm funding growth  :  152672.11509079937  et pvalue :  0.0
Expenses rev  :  0.008870004476197273  et pvalue :  1.0
Net int margin  :  1.5946386173188956  et pvalue :  0.0
Net loans total assets  :  2.456768872354506  et pvalue :  0.0
Non earning assets gs over TA  :  0.5649017092173558  et pvalue :  1.0
Operating roaa  :  0.37972170250782383  et pvalue :  1.0
Tax expense over TA  :  0.6880081896376785  et pvalue :  1.0
Total desposits over TA  :  2.034831118157987  et pvalue :  0.0
Total funding over TA  :  2.5963216089481618  et pvalue :  0.0
Total int exp over TA  :  1.2895919397905415  et pvalue :  0.0
Total non earning assets over TA  :  0.5649017092150294  et pvalue :  1.0
Total oper expense over TA  :  1.23347675783805  et pvalue :  0.0
log_Avg earning assets over TA  :  0.6613811781727749  et pvalue :  1.0
log_Bus volume over TA  :  15.80961547912108  et pvalue :  0.0
log_Liquid assets deposits mm funding  :  1.24311111552138  et pvalue :  0.0
log_Total assets  :  2.338539289628786  et pvalue :  0.0
log_Total non int exp over TA  :  1.0376794097286892  et pvalue :  0.29
total_assets  :  22015.641606632307  et pvalue :  0.0
```

Scatter plots: combined effects

- dataframe of the distance between barycenter of scatter plots as described as bellow
- scatter plots:
  - a regulatory variable with all other features
  - in red: banks that went default
  - in blue: banks that didn't went default
- two points:
  - in cyan: median of the two variables in the sample with no default
  - in green: same with default
- euclidian distance: distance between the two points $\Rightarrow$ the higher the distance, the greater the explanatory power of the two variables together
- this euclidian distance is then normalize between 0 and 1 for comparison puposes.

It should appear that variables for which the means and variances in the two sample are significantly different will be relevant in splitng the data when combined. However it is of interest to measur it.

Distances and combined explanatory power

In [58]:
```
# define a function of "barycenter"
"""
The function returns the distance between two points :
    - the points of coordinates: [ median of main variable for non default ;
                                   median of secondary variable for non default ]
    - the points of coordinates: [ median of main variable for default ;
                                   median of secondary variable for default ]

"""


def distance(df, main_var, var) :
    a = np.median(df[(df["Dum_Default"]==0) & (df[main_var]>0)][[main_var]].apply(np.log))
    b = np.median(df[(df["Dum_Default"]==0) & (df[var]>0)][[var]].apply(np.log))
    c = np.median(df[(df["Dum_Default"]==1)& (df[main_var]>0)][[main_var]].apply(np.log))
    d = np.median(df[(df["Dum_Default"]==1)& (df[var]>0)][[var]].apply(np.log))

    return np.sqrt((a-c)**2 + (b-d)**2)
```

In [60]:
```
# calculation of the distance
d_distance_scatter = {}

for main_var in l_exo_var :
    d_distance_scatter[main_var] = []
    for var in l_exo_var :
        d_distance_scatter[main_var].append(distance(df, main_var, var))


# squared matrix stocked in a dataframe (as a correlation matrix)
"""
One point corresponds to the distance between the two scatter plots.
Can be interpreted as the explanatory power of the combination of the two variables.
On the diagonal: the combination aof a variable with itself => its own explanatory potential power
"""
df_distances = pd.DataFrame(d_distance_scatter, columns=l_exo_var)
df_distances.index = l_exo_var
```

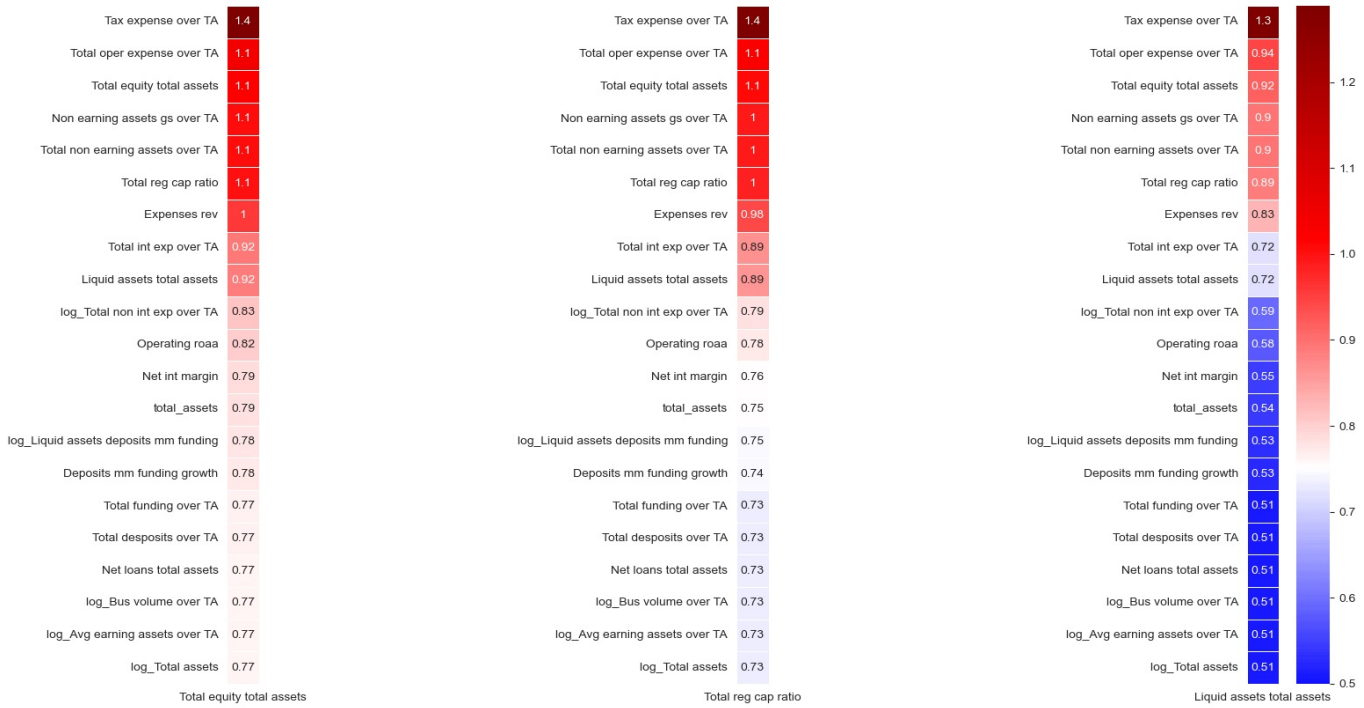
In [61]:
```
df_distances
```

Out[61]:

	Total equity total assets	Total reg cap ratio	Liquid assets total assets	Deposits mm funding growth	Expenses rev	Net int margin	Net loans total assets	Non earning assets gs over TA	Operating roaa	Tax expense over TA
Total equity total assets	1.083026	1.055926	0.918019	0.778713	1.008057	0.793938	0.766696	1.064746	0.815261	1.411164
Total reg cap ratio	1.055926	1.028112	0.885886	0.740560	0.978884	0.756552	0.727913	1.037168	0.778900	1.390474
Liquid assets total assets	0.918019	0.885886	0.715943	0.525556	0.828247	0.547861	0.507580	0.896380	0.578330	1.288875
Deposits mm funding growth	0.778713	0.740560	0.525556	0.199608	0.670543	0.252559	0.145851	0.753082	0.313210	1.193664
Expenses rev	1.008057	0.978884	0.828247	0.670543	0.927045	0.688165	0.656549	0.988391	0.712661	1.354481
Net int margin	0.793938	0.756552	0.547861	0.252559	0.688165	0.296190	0.212638	0.768814	0.349346	1.203651
Net loans total assets	0.766696	0.727913	0.507580	0.145851	0.656549	0.212638	0.051976	0.740649	0.282009	1.185859
Non earning assets gs over TA	1.064746	1.037168	0.896380	0.753082	0.988391	0.768814	0.740649	1.046146	0.790816	1.397183
Operating roaa	0.815261	0.778900	0.578330	0.313210	0.712661	0.349346	0.282009	0.790816	0.395420	1.217822
Tax expense over TA	1.411164	1.390474	1.288875	1.193664	1.354481	1.203651	1.185859	1.397183	1.217822	1.676252
Total desposits over TA	0.767265	0.728513	0.508440	0.148813	0.657214	0.214681	0.059787	0.741238	0.283553	1.186227
Total funding over TA	0.768260	0.729560	0.509940	0.153860	0.658375	0.218210	0.071428	0.742268	0.286234	1.186871
Total int exp over TA	0.920966	0.888940	0.719718	0.530687	0.831513	0.552785	0.512891	0.899398	0.582997	1.290975
Total non earning assets over TA	1.064746	1.037168	0.896380	0.753082	0.988391	0.768814	0.740649	1.046146	0.790816	1.397183
Total oper expense over TA	1.105747	1.079218	0.944718	0.810016	1.032430	0.824663	0.798471	1.087849	0.845212	1.428676
log_Avg earning assets over TA	0.765884	0.727057	0.506352	0.141516	0.655600	0.209689	0.038155	0.739808	0.279792	1.185334
log_Bus volume over TA	0.766515	0.727722	0.507306	0.144892	0.656337	0.211982	0.049222	0.740461	0.281515	1.185742
log_Liquid assets deposits mm funding	0.783504	0.745595	0.532628	0.217548	0.676101	0.266964	0.169577	0.758034	0.324937	1.196794
log_Total assets	0.765883	0.727057	0.506352	0.141516	0.655600	0.209689	0.038154	0.739808	0.279792	1.185334
log_Total non int exp over TA	0.826101	0.790239	0.593513	0.340429	0.725036	0.373945	0.311964	0.801986	0.417312	1.225105
total_assets	0.790458	0.752900	0.542806	0.241397	0.684147	0.286732	0.199253	0.765220	0.341364	1.201358

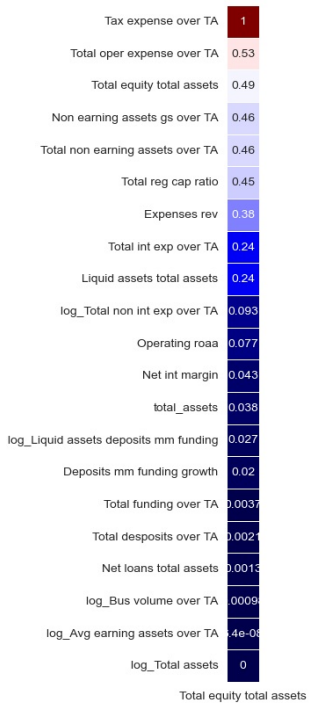
21 rows × 21 columns

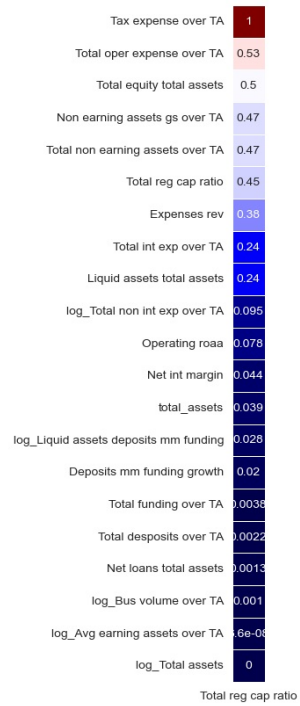
In [63]:
```
# setting the plot
fig, axs = plt.subplots(ncols=3, gridspec_kw=dict(width_ratios=[1.5,1,1]))


#Plotting cmd
var = "Total equity total assets"
sns.heatmap(df_distances.iloc[:,0:1].sort_values(var,ascending=False),ax=axs[0], cmap=cmap,cbar=False, center=.
            square=True, vmin=.5, linewidths=.5, annot=True)


var = "Total reg cap ratio"
sns.heatmap(df_distances.iloc[:,1:2].sort_values(var,ascending=False),ax=axs[1],cbar=False, cmap=cmap, center=.
            square=True, vmin=.5, linewidths=.5, annot=True)

var = "Liquid assets total assets"
sns.heatmap(df_distances.iloc[:,2:3].sort_values(var,ascending=False),ax=axs[2], cmap=cmap, center=.75,
            square=True, vmin=.5, linewidths=.5, annot=True)

# plt.savefig(OUTPUT_PATH + 'distances_barycenters_bivar.png')


plt.show()
```

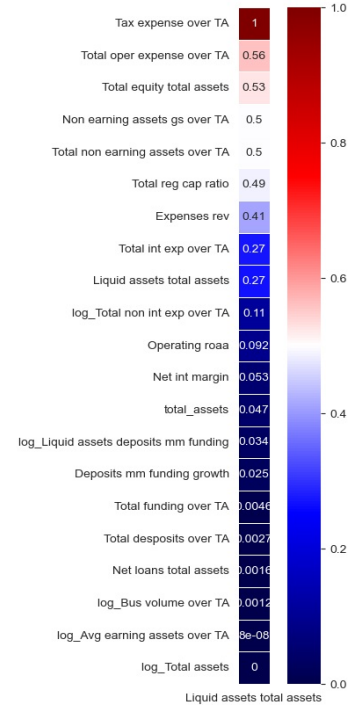


The following plot gives ordered heatmaps of the min-max normilzed distances between each of our variables of main interest (regulatory variables) and all the others. The dstance is computed as the eucldian distance between the scatter plots' barycenters.

In [64]:
```
# normalize the distance dataframe
normalized_df=(df_distances-df_distances.min())/(df_distances.max()-df_distances.min())

# setting the plot
fig, axs = plt.subplots(ncols=3, gridspec_kw=dict(width_ratios=[1.5,1,1]))


#Plotting cmd
var = "Total equity total assets"
sns.heatmap(normalized_df.iloc[:,0:1].sort_values(var,ascending=False),ax=axs[0], cmap=cmap,cbar=False, center=
            square=True, vmin=0, linewidths=.5, annot=True)


var = "Total reg cap ratio"
sns.heatmap(normalized_df.iloc[:,1:2].sort_values(var,ascending=False),ax=axs[1],cbar=False, cmap=cmap, center=
            square=True, vmin=0, linewidths=.5, annot=True)

var = "Liquid assets total assets"
sns.heatmap(normalized_df.iloc[:,2:3].sort_values(var,ascending=False),ax=axs[2], cmap=cmap, center=.5,
            square=True, vmin=0, linewidths=.5, annot=True)

# plt.savefig(OUPUT_PATH + 'distances_barycenters_bivar.png')


plt.show()
```



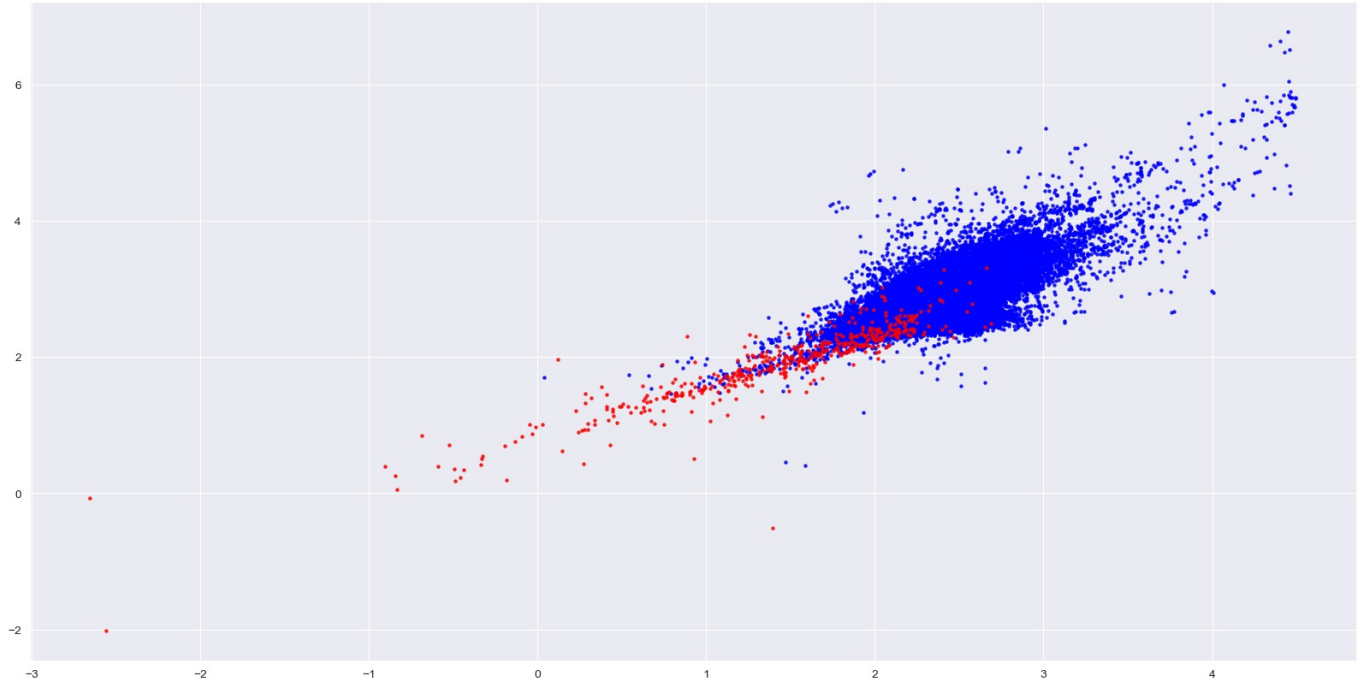




Combined relevance: scatter plots

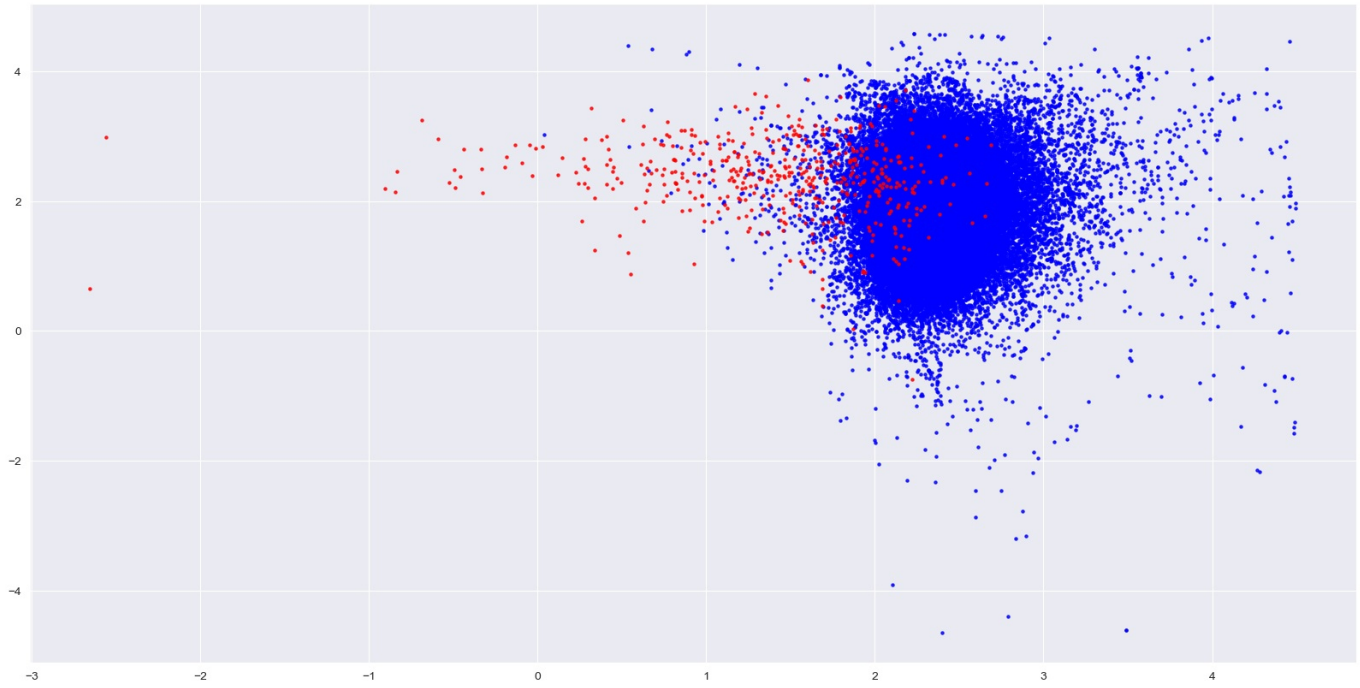
In [65]:
```
plt.scatter(np.log(df[df["Dum_Default"]==0]["Total equity total assets"]),
            np.log(df[df["Dum_Default"]==0]["Total reg cap ratio"]),
           s=5, c="b",alpha=0.8)

plt.scatter(np.log(df[df["Dum_Default"]==1]["Total equity total assets"]),
            np.log(df[df["Dum_Default"]==1]["Total reg cap ratio"]),
           s=5, c="r",alpha=0.8)

plt.show()
```

```
C:\Users\pipsou\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning: invalid value encounte
red in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
```



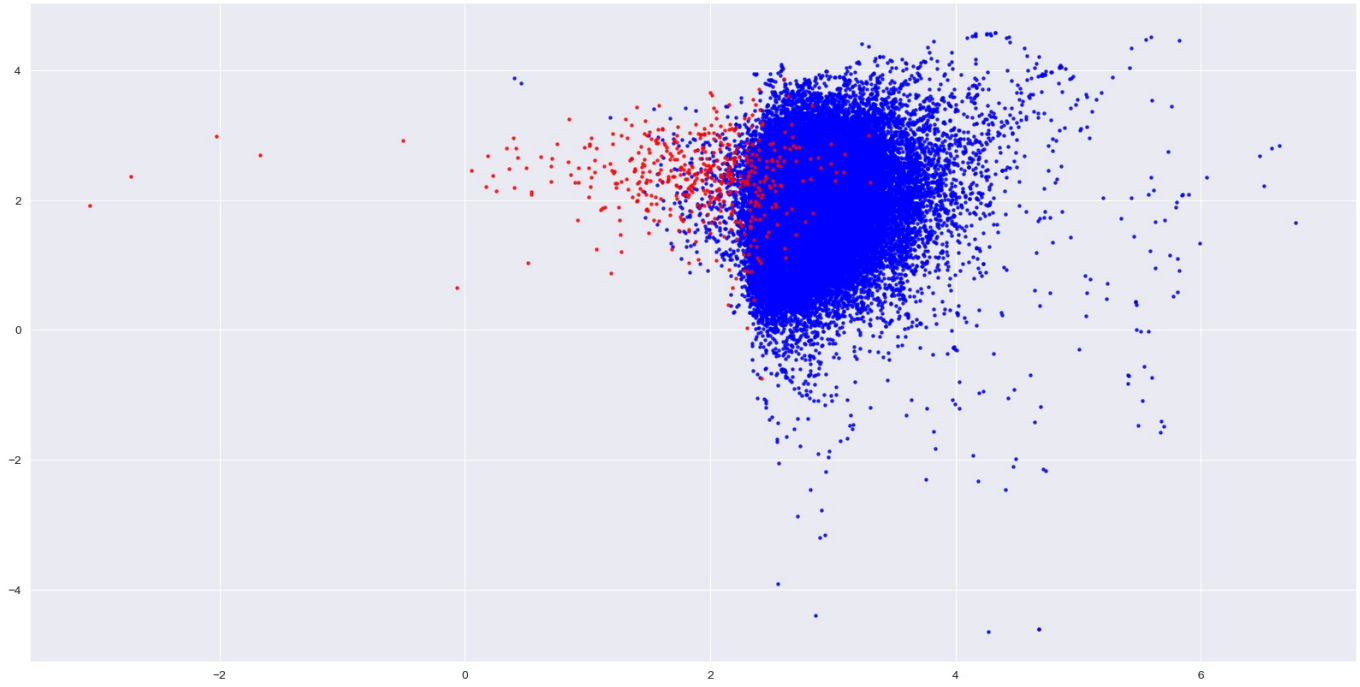
In [66]:
```
plt.scatter(np.log(df[df["Dum_Default"]==0]["Total equity total assets"]),
            np.log(df[df["Dum_Default"]==0]["Liquid assets total assets"]),
           s=5, c="b",alpha=0.8)

plt.scatter(np.log(df[df["Dum_Default"]==1]["Total equity total assets"]),
            np.log(df[df["Dum_Default"]==1]["Liquid assets total assets"]),
           s=5, c="r",alpha=0.8)

plt.show()
```

```
C:\Users\pipsou\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning: invalid value encounte
red in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
```



In [67]:
```
plt.scatter(np.log(df[df["Dum_Default"]==0]["Total reg cap ratio"]),
            np.log(df[df["Dum_Default"]==0]["Liquid assets total assets"]),
           s=5, c="b",alpha=0.8)

plt.scatter(np.log(df[df["Dum_Default"]==1]["Total reg cap ratio"]),
            np.log(df[df["Dum_Default"]==1]["Liquid assets total assets"]),
           s=5, c="r",alpha=0.8)

plt.show()
```

```
C:\Users\pipsou\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning: invalid value encounte
red in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
```



Two way scatter-density plots (SMOTE sample)

In [68]:
```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
```

In [69]:
```
# Defining features and labels matrices
X = df[list(df)[6:-3]]
y = df[list(df)[5]]
```

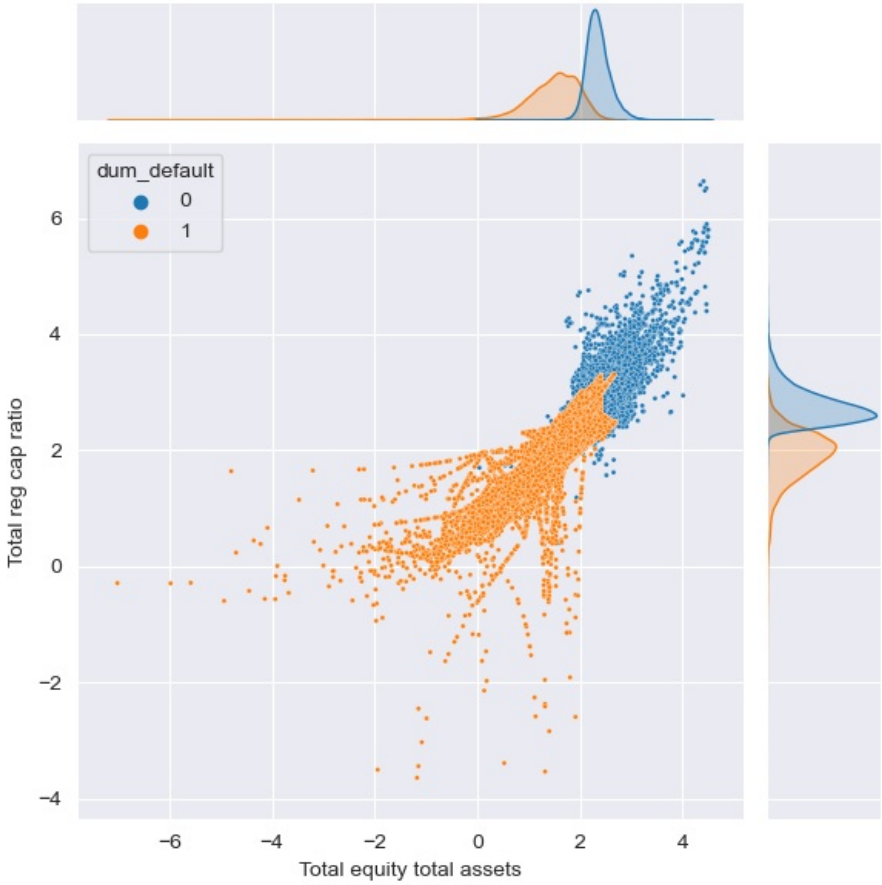
```
# Applying SMOTE procedure on training data
smote = SMOTE(random_state=601)
X_smote, y_smote = smote.fit_resample(X_train, y_train)
```

In [70]:
```
data = X_smote
data["dum_default"] = y_smote
```

In [71]:
```
sns.jointplot(data = data,
              x=np.log(data["Total equity total assets"]),
              y=np.log(data["Total reg cap ratio"]),
              hue="dum_default" ,
              kind='scatter', s=5)
```

```
C:\Users\pipsou\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning: invalid value encounte
red in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
```

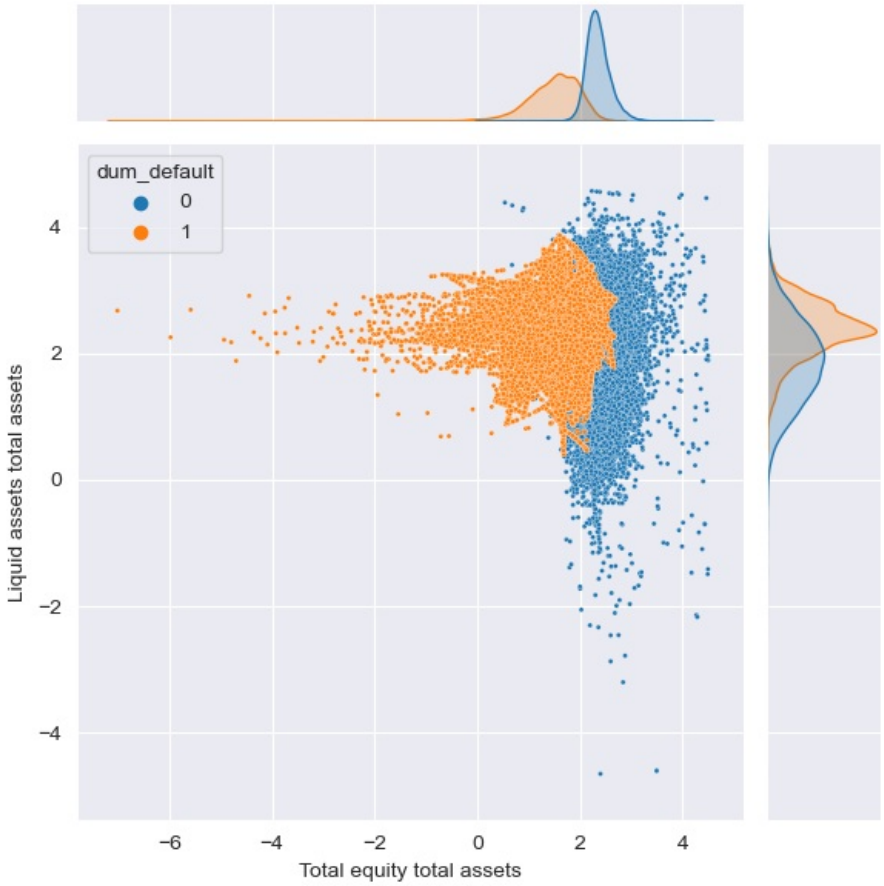
Out[71]: <seaborn.axisgrid.JointGrid at 0x263d05c9e50>



In [72]:
```
sns.jointplot(data = data,
              x=np.log(data["Total equity total assets"]),
              y=np.log(data["Liquid assets total assets"]),
              hue="dum_default" ,
              kind='scatter', s=5)
```

```
C:\Users\pipsou\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning: invalid value encounte
red in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
```

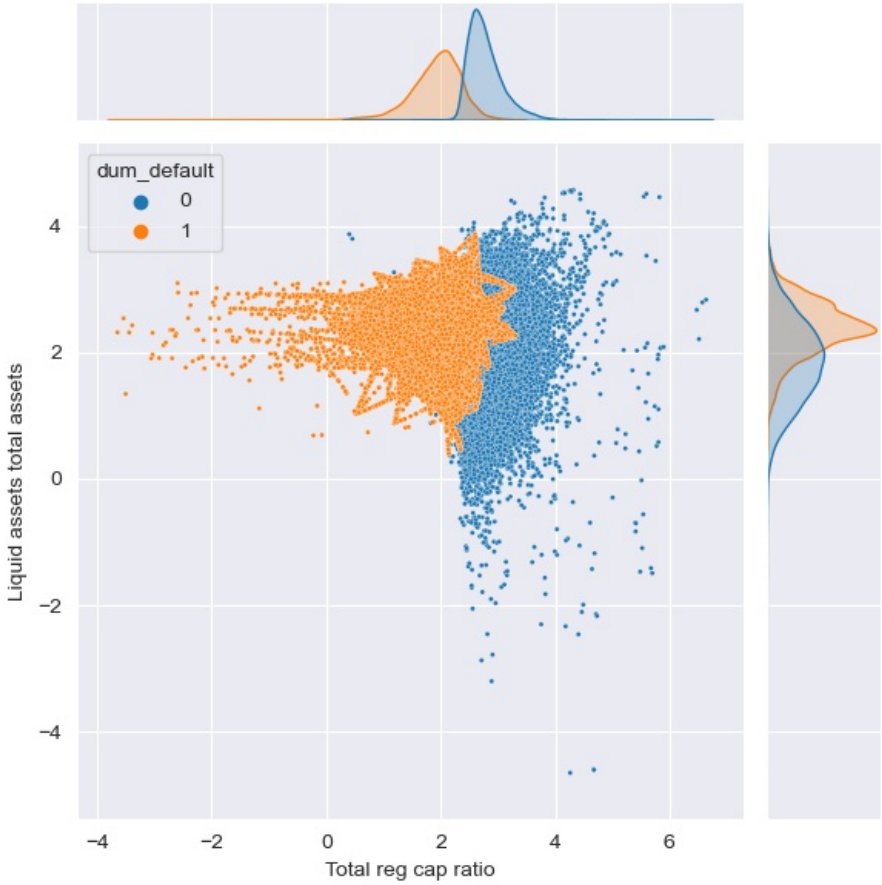
Out[72]: <seaborn.axisgrid.JointGrid at 0x263d3d14820>



In [73]:
```
sns.jointplot(data = data,
              x=np.log(data["Total reg cap ratio"]),
              y=np.log(data["Liquid assets total assets"]),
              hue="dum_default" ,
              kind='scatter', s=5)
```

```
C:\Users\pipsou\anaconda3\lib\site-packages\pandas\core\arraylike.py:397: RuntimeWarning: invalid value encounte
red in log
  result = getattr(ufunc, method)(*inputs, **kwargs)
```

Out[73]: <seaborn.axisgrid.JointGrid at 0x263cc38c5b0>



Default banks evolution in years before default

In [79]:
```
l_def_bk = df[df.Dum_Default == 1].value_counts
```

In [87]:
```
df[df.Dum_Default == 1].ID.value_counts().index
```

Out[87]:
```
Int64Index([18105870, 10870371,  7241868,  3621153, 25266801,  3609189,
             2415076,  8418921,  4811888,  2406780,
            ...
             1203061, 28829232,  1207790, 25351263,  2402524,  1066383,
             1039827,  1069604,  1202851, 26531362],
           dtype='int64', length=459)
```

In [ ]: